Locality and Distance-Aware Attention for Time Series Extrinsic Regression

Joshua Fan*[1], Kaitlyn Chen*[1], Shufeng Kong[1], Junwen Bai[1], Zhiyun Li[3], Ariel Ortiz-Bobea[2], Carla P. Gomes[1]

[1]Cornell University, Department of Computer Science
[2]Cornell University, Charles H. Dyson School of Applied Economics and Management
[3]UCLA, Anderson School of Management
{jyf6,kjc256,sk2299,jb2467}@cornell.edu,zhiyun.li@anderson.ucla.edu,ao332@cornell.edu, gomes@cs.cornell.edu

Abstract

Time-series data play a crucial role in domains such as agriculture, environmental science, and healthcare. A common task in this context is *time series extrinsic regression,* which requires mapping a multivariate sequence to a single numeric output. While Transformers excel in language by modeling semantic relationships via self-attention, we demonstrate that standard self-attention is poorly suited for modeling temporal relationships. It ignores critical aspects like temporal distance and locality, while adding unnecessary complexity. Surprisingly, simply removing self-attention and processing timesteps independently with an MLP already improves performance, highlighting self-attention's negative impact in this context. More broadly, models that first learn local features (akin to convolutions) before global aggregation seem effective. To bridge this gap, we propose Locality And Distance-Aware Attention (LADAA), which explicitly encodes this temporal structure. In LADAA, attention scores depend solely on relative distance in a smooth way, and are initialized to prioritize local features while retaining flexibility to model long-range relationships. LADAA achieves state-of-the-art results on multiple time series extrinsic regression benchmarks, outperforming Transformer-based, convolutional, and recurrent neural networks. It also significantly improves performance on a challenging crop yield prediction task.

1 Introduction

Time-series extrinsic regression, the task of mapping multivariate time-series to a single numeric value (Tan et al. 2021), presents unique challenges that distinguish it from other time-series problems. Unlike forecasting (predicting the same variables at a future timestep) or classification (learning several decision boundaries), time-series extrinsic regression requires distilling complex temporal patterns across multiple variables into a single value, while preserving sensitivity to critical local events. This capability is vital across domains: in agriculture, predicting annual crop yields from daily weather data requires learning nonlinear dependencies on specific growing-season conditions (Ortiz-Bobea 2021); in energy management, building consumption hinges on correlating occupancy patterns with weather fluctuations (Candanedo, Feldheim, and Deramaix 2017); and in environmental health, pollution levels emerge from compounding effects of industrial activity and meteorological shifts (De Vito et al. 2008). Yet despite its applicability, methodological development for time series extrinsic regression lags behind other time-series tasks, with most approaches adapting models designed for other tasks (Tan et al. 2021).

Transformers have revolutionized sequential modeling in natural language processing by effectively capturing both the semantic and syntactic structure of language through self- attention (Vaswani et al. 2017). Since time series are also sequences, several works treat timesteps as "tokens" and apply a vanilla Transformer on them (Zerveas et al. 2021; Chowdhury et al. 2022). However, Transformers rely on feature similarity to compute attention scores, ignoring temporal locality and distance, which are critical for time-series reasoning. Crucially, timesteps lack inherent semantics; timesteps with similar features may not be dependent.

Surprisingly, we show that *removing self-attention entirely* can improve performance on time series extrinsic regression. Simply applying a multilayer perceptron (MLP) on each timestep and pooling the outputs already outperforms full Transformers on multiple datasets. In crop yield prediction, CNN-based methods also outperform Transformers. This indicates that self-attention is not learning meaningful interactions between timesteps, and can even hurt performance by overfitting. This aligns with findings in forecasting, where linear models surpass complex attention variants (Zeng et al. 2023). Self-attention tends to diffuse focus globally, instead of learning localized features, which are important in time series (Dempster, Petitjean, and Webb 2020).

To bridge this gap, we draw inspiration from convolution's inductive bias for locality and position sensitivity, which has been effective for time series (Dempster, Petitjean, and Webb 2020). While prior work in time series classification injects relative distances alongside content-based attention (Foumani et al. 2024), we propose a radical simplification: attention scores should depend *exclusively* on temporal distance. Our method, *Locality and Distance-Aware Attention (LADAA)*, eliminates content-based interactions entirely. LADAA initializes attention to prioritize local windows (emulating convolutional locality), then dynamically expands receptive fields to learn longer-range dependencies if needed. LADAA outperforms a wide range of

*These authors contributed equally.

baseline methods on multiple time series extrinsic regression datasets, including a challenging crop yield forecasting task.

Our **main contributions** can be summarized as follows:

1. We show that a "per-timestep MLP" with attention pooling already outperforms standard Transformers on multiple time series extrinsic regression tasks. This raises questions about whether content-based self-attention is appropriate for time series.
2. We then propose *LADAA*, where attention scores only depend on relative distances in a smooth way (except for a global pooling attention at the end). We develop an initialization scheme that encourages each timestep to pay most attention to its neighbors, while allowing for flexibility if necessary.
3. We perform an extensive evaluation of our approaches on time series extrinsic regression datasets, including a challenging crop yield prediction task. LADAA outperforms a diverse range of methods, including Transformer, convolutional, and classical models, reducing RMSE by an average of 18 percent over the best baseline.

2 Background and Related Work

Time series extrinsic regression. This paper focuses on *time series extrinsic regression* (Tan et al. 2021), which aims to learn a mapping between a multivariate time series $\mathbf{X} \in \mathbb{R}^{T \times V}$ (V variables at T timesteps) to a number $y \in \mathbb{R}$. Time series extrinsic regression is related to the tasks of time series classification (where y is a discrete class), and time series forecasting (where y is the same variables as $\mathbf{X}$ at a future timestep), but is less studied. While deep learning is clearly state-of-the-art for classification and forecasting, classical methods that ignore temporal structure remain competitive in time series extrinsic regression (Tan et al. 2021). Before Transformers were adopted, the best method was ROCKET (Dempster, Petitjean, and Webb 2020), which generates many random convolutional kernels with different receptive fields, and then trains a linear model on top of the extracted features. Several deep learning models have also been proposed; see Mohammadi Foumani et al. (2024) for a survey. A strong approach is convolutional neural networks, which slide learned convolutions along the time series; for example, InceptionTime aggregates filters of different lengths (Ismail Fawaz et al. 2020). These approaches often perform well but have high variance (Tan et al. 2021).

Transformers for time series. Given the success of the Transformer architecture in processing text, it is natural to apply them on time series, as both are sequential modalities. Zerveas et al. (2021) were among the first to do this. They view each timestep as a token, with its features as its embedding. Then they use standard self-attention, which we review for completeness. Let $\mathbf{x}_i \in \mathbb{R}^{d_x}$ be the embedding of timestep i. For each head h and timestep i, we compute three linear projections of the embedding:

$$\mathbf{q}_i^h = \mathbf{x}_i W_Q^h,\ \mathbf{k}_i^h = \mathbf{x}_i W_K^h,\ \mathbf{v}_i^h = \mathbf{x}_i W_V^h \tag{1}$$

where $W_Q^h, W_K^h, W_V^h \in \mathbb{R}^{d_x \times d_h}$ are learnable matrices (per layer and head).

Now, at each timestep i (and head h), we compute a weighted sum over *all* timesteps' values:

$$\mathbf{z}_i^h = \sum_{j=1}^{n} \alpha_{ij}^h \mathbf{v}_j^h \tag{2}$$

Then we concatenate each head's output, and pass it through a feedforward network (MLP). We will now look at a single head, remove h for clarity, and focus on α_{ij}, which is an *attention score* that indicates how much timestep i should pay attention to timestep j. It is computed with a softmax:

$$content\_sim(i,j) = \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp(e_{ik})} \tag{3}$$

where e_{ij} is the scaled dot product between the query of timestep i and key of timestep j:

$$e_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_z}} \tag{4}$$

Note that the attention between timesteps i and j only depends on their features, not their distance or position.

Many variants of this architecture have been proposed for the related tasks of time series *forecasting* and *classification*, such as Informer (Zhou et al. 2021), Autoformer (Wu et al. 2021b), Pyraformer (Liu et al. 2022), FEDformer (Zhou et al. 2022), Crossformer (Zhang and Yan 2023), and Gated Transformer Networks (Liu et al. 2021a). These works often aim to make the attention mechanism more efficient or sparse, or add multi-resolution and hierarchical structure; see Wen et al. (2023) for a survey. However, Zeng et al. (2023) found that simple linear models can often outperform complex Transfomer-based architectures, leading to questions about whether self-attention is suited to processing time series. While follow-up works were able to beat the linear baseline, they operate on multivariate time series in a highly simplified way; PatchTST (Nie et al. 2023) and CATS (Kim et al. 2024) treat each channel as independent and use patching to enhance locality, while Inverted Transformer (Liu et al. 2024) only applies attention across variables, not across time.

Positional Encoding. The self-attention operation views sequences as an *unordered* set of time points, ignoring their position. In the original Transformer paper (Vaswani et al. 2017), the only way that positional information is retained is the *positional encoding*, which is a fixed vector $\mathbf{p}_i$ for each position determined by sinusoidal functions. This is added to the input embedding of each timestep: $\mathbf{x}_i = \mathbf{x}_i + \mathbf{p}_i$. While this scheme causes nearby timesteps to have closer embeddings, it does not directly encode relative distances into the attention mechanism. Zerveas et al. (2021) used a learnable vector for each position, which also does not explicitly encode relative distance information.

The first work to use *relative positions* to compute attention scores was (Shaw, Uszkoreit, and Vaswani 2018), which add a vector $\mathbf{w}_{i-j} \in \mathbb{R}^{d_h}$ to the key, modifying Eq. 4 to:

$$e_{ij} = \frac{(\mathbf{q}_i) \cdot (\mathbf{k}_j + \mathbf{w}_{i-j})}{\sqrt{d_z}} \tag{5}$$

A vector $\mathbf{w}_{i-j}$ is learned for each offset $(i-j)$ within a range $[-k, k]$ (offsets outside this range are clipped).

Foumani et al. (2024) simplify this into *efficient Relative Positional Encoding (eRPE)*, where we learn a single scalar w_{i-j} for each possible offset $(i-j)$. This modifies Eq. 3 to

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T} \exp(e_{ik})} + w_{i-j} \tag{6}$$

The bias is added outside the softmax, making the position values sharper.

Locality inductive biases in Transformers. Some works aim to inject convolutional inductive biases into Transformers, to make them more locality-aware and hierarchical. Pyraformer (Liu et al. 2022) constructed a hierarchical pyramidal attention scheme that computes fine-scale features and aggregates them, but performed worse than a linear baseline (Zeng et al. 2023). PatchTST used patching to construct tokens from each variable's time-series, which is similar to a convolutional layer (Nie et al. 2023). In computer vision and audio (which also have continuous spatial/temporal structure), some works have integrated convolutions and Transformers. Convolution-Augmented Transformer (Gulati et al. 2020), Conformer (Peng et al. 2021), and AlterNet (Park and Kim 2022) proposed mixed architectures combining convolution and multihead self-attention layers. CvT added a convolutional token embedding and used convolutional projections so that the query/key/value at time i depends on i's neighbors (Wu et al. 2021a). CMT added a depthwise convolution before the self-attention layer, and added a pre-softmax relative positional encoding bias (Guo et al. 2022). Swin Transformers restrict self-attention to be within local windows, and shift windows in later layers to allow information to spread (Liu et al. 2021b). Compact Convolutional Transformers also used a convolutional tokenizer and a attention pooling layer to aggregate tokens into a global prediction (Hassani et al. 2021). ConViT embed convolutional inductive biases via the relative positional embedding (d'Ascoli et al. 2021), which we discuss in Methods.

3 Methods

This section presents our Locality and Distance-Aware Attention (LADAA) framework for time series extrinsic regression. We first establish that a simple Per-Timestep MLP already outperforms standard Transformers, showing that content-based attention does not learn useful interactions between timesteps. We then introduce our novel Locality and Distance-Aware Attention mechanism, which computes attention scores purely based on temporal distances, eliminating content-based interactions. We initialize these attention scores for convolution-like locality while maintaining global flexibility. Finally, we add smoothness and sparsity regularization to ensure coherent and smooth attention over time.

3.1 Per-timestep MLP

We first show that processing each timestep independently with an MLP, followed by attention pooling, can outperform a full Transformer on many time series extrinsic regression datasets. This is motivated by agricultural domain knowledge; crop yields can be predicted using the sum of daily temperature effects (Ortiz-Bobea 2021). Agricultural scientists often construct features such as Growing Degree Days as the sum of daily effects (Butler and Huybers 2015), e.g.:

$$GDD = \sum_{t \in \text{season}} f(\mathbf{x}_t) \tag{7}$$

where $f(\mathbf{x}_t)$ is a function of the temperature on day t, e.g.

$$f(T) = \min(\max(T - 10, 0), 20) \tag{8}$$

Thus, a model that computes an effect for each timestep independently and finally adds together the effects makes sense according to agricultural domain knowledge. We therefore propose a lightweight "per-timestep MLP":

1. **Patching (optional):** For long series, we apply a patching operation to generate tokens. If $[\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T]$ is the original time series, we compute patches as
$$[\mathbf{x}_{1:P}; \mathbf{x}_{1+S:P+S}; \mathbf{x}_{1+2S:P+2S}; \ldots] \tag{9}$$
In other words we generate tokens from flattened feature vectors for a patch with P timesteps, spread S apart.
2. **Per-timestep MLP:** We then pass each input token through an MLP: $\mathbb{R}^{VP} \to \mathbb{R}^d$ to obtain an embedding:
$$\mathbf{h}_t = MLP(\mathbf{x}_t; \theta) \tag{10}$$
All timesteps share the same MLP, but are processed independently. The MLP contains 3 layers consisting of BatchNorm1d, ReLU, and Linear operations.
3. **Absolute positional encoding:** We concatenate a positional encoding $\mathbf{p}_t \in \mathbb{R}^d$ to each timestep's embedding:
$$\mathbf{h}_t = concat(\mathbf{h}_t, \mathbf{p}_t) \tag{11}$$
$\mathbf{p}_t$ are initialized from sinuosoidal functions (Vaswani et al. 2017), but are further optimized in training.
4. **Attention pooling:** Similar to (Bahdanau 2014; Hassani et al. 2021), we use attention pooling to aggregate the embeddings from each timestep into a global embedding, which is then used to make a prediction:
$$\mathbf{h}_{global} = \sum_{i=1}^{T} \alpha_i \mathbf{h}_i; \quad \alpha_i = softmax(MLP_{attn}(\mathbf{h}_i)) \tag{12}$$
Since $\mathbf{h}_i$ contains content and positional information, this can perform a weighted average of timesteps that are relevant because of their position or features. It can also express global max or average pooling (see Appendix).

Surprisingly, this model already outperforms a full Transformer with multi-head self attention (see Results). The success of this baseline indicates that in many time series extrinsic regression datasets, we can decompose the label into a sum of effects of each timestep, and that *local-global* structure is efffective (initial layers focus on local features, before a final global aggregation).

But why do vanilla Transformers perform poorly? Looking at example attention matrices in Figure 1, we see that attention matrices in a standard Transformer often have rows

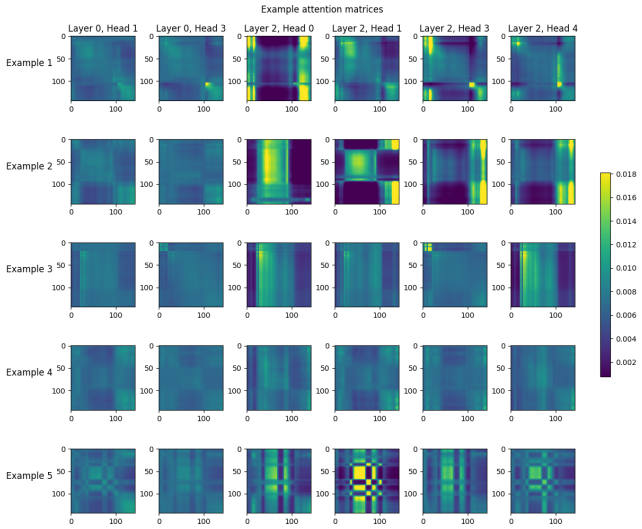

Figure 1: Selected attention matrices from a vanilla Transformer run. Many of the matrices are uniform, or have identical rows (meaning that all timesteps pay attention to the exact same timesteps. Other matrices show block patterns similar to the similarity matrix between timesteps' features. We do not see any diagonal patterns, which we would have seen if timesteps were paying attention to their neighbors.

that are nearly identical, meaning that each timestep is receiving the exact same information from all other timesteps. While the residual connection allows each timestep to preserve some local information, the attention mechanism is just adding a constant to each timestep and is not meaningfully learning interactions between timesteps.

3.2 Locality and Distance-Aware Attention

While the per-timestep MLP already does well, it ignores interactions between timesteps. We want to strategically add back some interactions in step (2). Recall that in standard attention, the score between timesteps i and j is based on the dot product of their query/key:

$$\alpha_{ij} = softmax_j\left(\frac{\mathbf{q_i} \cdot \mathbf{k_j}}{\sqrt{d}}\right) = content\_sim(i,j) \quad (13)$$

Essentially, each timestep pays attention to timesteps with similar features.[1] In text, this makes sense as the relationship between words depends on their semantic meaning more than their position. However, in time series, there is little semantic meaning to each timestep's features.

As discussed, *relative positional encodings* make attention scores depend on the *temporal distance* between timesteps. For example, ConViT computes a *positional attention score* that that depends on the *offset* between pixels i, j. Adapting it to time series, each head learns a *center of attention* Δ^h and *locality strength* α^h, and the positional attention is computed as

$$pos\_sim(i,j) = softmax(-\alpha^h \cdot |\Delta^h - (i-j)|) \quad (14)$$

Note that this attention score is a function of the relative distance between timesteps $(i-j)$. In ConViT, we assume each head focuses its attention on a specific offset Δ^h: the (pre-softmax) attention weight is highest at that offset, and decays linearly from there.

If we do not want to restrict ourselves to linear decays from a peak, inspired by (Foumani et al. 2024) we can learn a bias table $\mathbf{B} \in \mathbb{R}^{2T-1 \times H}$, where for each offset $(i-j)$ and head we have a scalar that indicates how much extra attention should be paid to the timestep at that offset.

Note that there are multiple possible ways to combine content and positional attention scores. ConViT (d'Ascoli et al. 2021) used a gating mechanism:

$$\alpha^h_{ij} = (1-\lambda)\cdot content\_sim(i,j) + \lambda \cdot pos\_sim(i-j) \quad (15)$$

On the other hand, Foumani et al. (2024) simply added the

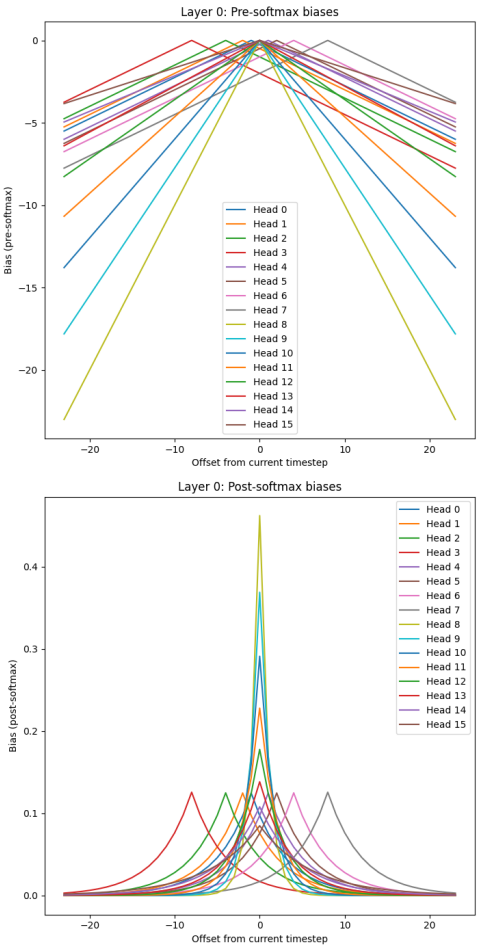

Figure 2: Initial relative positional biases: before softmax (left), after softmax (right). X-axis is offset from current timestep (0 at middle), Y-axis is bias value.

[1] While $\mathbf{q_i}, \mathbf{k_j}$ have information about their position from the positional embedding, Figure 1 suggests they are not learning relative offsets from it – otherwise we would see more diagonal patterns.

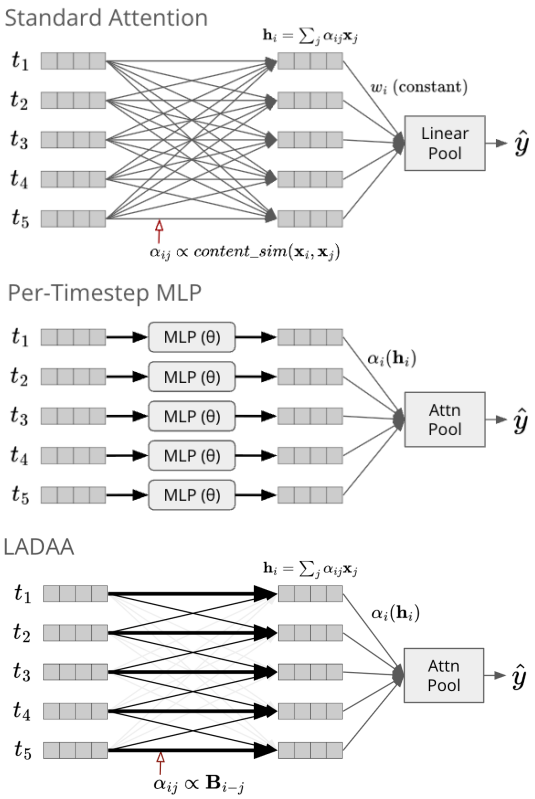

Figure 3: Schematic comparing attention patterns. Top is standard attention (each timestep pays attention to all other timesteps, based on their feature similarity). Middle is per-timestep MLP. Bottom is LADAA with convolutional initialization (each timestep pays attention to all other timesteps, based on their temporal distances, but is initialized to focus on nearby timesteps).

relative positional bias to the content attention score:

$$\alpha_{ij}^h = content\_sim(i,j) + pos\_sim(i-j) \tag{16}$$

However, we find that for time series, we can completely *remove* content attention and rely purely on relative positional attention. As in (Foumani et al. 2024), for each layer we learn a bias table for each possible offset $(i-j)$: $\mathbf{B} \in \mathbb{R}^{2T-1 \times H}$. Then, the attention matrix is simply computed by passing these biases through a softmax:

$$\alpha_{ij}^h = \frac{\exp(\mathbf{B}_{i-j,h})}{\sum_{k=1}^{T} \exp(\mathbf{B}_{i-k,h})} \tag{17}$$

Following (Cordonnier, Loukas, and Jaggi 2019), it is straightforward to show that an "Only Relative Positional Encoding" layer can behave like any convolutional layer:

Proposition 1. *For any 1D convolution with filter size h, an "Only Relative Positional Encoding" layer with h heads is capable of expressing the same function.*

Proof. Suppose we have a convolutional filter that at timestep i, takes inputs at positions $i-j$ for various offsets $j \in \mathcal{N}$. We simply make each attention head focus on a particular offset $(i-j)$, by setting $\mathbf{B}_{i-j}^h$ to a large value and $\mathbf{B}_{k \neq (i-j)} = -\infty$, so that after the softmax, the head has attention score 1 on that particular offset and 0 elsewhere. Thus, each head is simply extracting the features at timestep $i-j$. The value/output projections apply a linear function to the features at each of those offsets, which is exactly what a convolution does. Details can be found in Lemma 1 in (Cordonnier, Loukas, and Jaggi 2019) and the Appendix. □

However, if we want to encourage this convolutional behavior, we can initialize the bias table in a way that is close to the construction in Propositon 1. In this paper, we initialize half of the heads using ConViT linear decays (d'Ascoli et al. 2021): we specify a center of attention Δ^h and locality strength α^h for each head, and set the biases based on linear decays from the center:

$$\mathbf{B}_{i-j,h} = -\alpha^h \cdot |\Delta^h - (i-j)|$$

In this paper, we set $\alpha^h = 0.25$ and the centers of attention $\Delta = [-8, -4, -2, -1, 1, 2, 4, 8]$, to emulate a dilated convolution looking at those offsets. While this worked for all datasets considered here, learning an initialization that is better adapted to each dataset's temporal resolution may be helpful for future work.

We also initialize half the heads according to Alibi biases (Press, Smith, and Lewis 2021), meaning that they pay most attention to the current timestep and linearly decay from there, but at different slopes. Specifically, we compute 8 slopes as $s \in [2^0, \ldots, 2^{-\log_2(T/4)}]$ (by geometrically interpolating), and initialize the biases as

$$\mathbf{B}_{i-j,h} = -s_h \cdot |i-j|$$

See Figure 2 for a diagram of the initialized biases, and Figure 3 for a schematic showcasing the locality bias. We call our initialization "Convalibi init" as it combines ideas from ConViT and Alibi.

3.3 Smoothness and Sparsity Regularization

While we learn a bias for each discrete offset $(i-j)$, time series are continuous, so we want to learn a smooth attention function over time. While we could fit this function with an MLP (Li et al. 2023), here we keep the original parameterization, but add a smoothness loss that encourages attention for nearby timesteps to be similar (for each layer/head):

$$\mathcal{L}_{smooth} = \sum_{i=1}^{T} \sum_{j=1}^{T-1} |\alpha_{i,j} - \alpha_{i,j+1}| \tag{18}$$

We also do the same for pooling attention. Finally, we add L1 regularization on the weights of linear layers, to improve the model's sparsity (the model can completely ignore some attention heads) and robustness (if the input changes slightly, we do not want the output to change too much).

4 Experiments

4.1 Time Series Extrinsic Regression benchmarks

We first evaluate our techniques on the six time series extrinsic regression datasets used in (Zerveas et al. 2021), taken

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG
Random Forest	3.455	0.855	94.072	63.301	44.657	32.109
XGBoost	3.489	0.637	93.138	59.495	44.295	31.487
Rocket	2.299	3.36	120.057	62.769	41.829	36.515
Inception	4.435	1.584	96.749	62.227	51.539	*23.903*
TST	2.228	0.517	91.344	60.357	42.607	25.042
Per timestep MLP	*1.954 ± 0.133*	*0.277 ± 0.035*	89.363 ± 1.195	55.798 ± 0.999	41.138 ± 0.010	32.305 ± 0.952
LSTM + Attn	2.208 ± 0.104	0.460 ± 0.085	90.519 ± 0.957	57.059 ± 0.405	41.487 ± 0.217	24.784 ± 0.630
ConViT	2.398 ± 0.069	0.326 ± 0.107	*87.685 ± 0.789*	*54.759 ± 0.744*	42.842 ± 0.452	Out of memory
ConvTran	2.101 ± 0.264	0.536 ± 0.048	89.230 ± 0.610	56.847 ± 0.306	43.450 ± 0.745	Out of memory
LADAA	**1.685 ± 0.084**	**0.273 ± 0.034**	**86.878 ± 1.051**	**53.999 ± 1.215**	41.683 ± 0.839	**20.937 ± 0.140**

Table 1: Results (RMSE ↓) on Time Series Extrinsic Regression archive. Above the line are baseline results reported in (Zerveas et al. 2021). Below the line is our methods; we provide mean and standard deviation over 3 seeds. Best is **bolded**, second-best is *italicized*. Per-timestep MLP outperforms all above baselines for all datasets except IEEEPPG. LSTM+Attn outperforms all baselines. ConViT and Convtran are mixed but perform well overall. LADAA is the best on all datasets.

	RMSE (trend deviation, ↓)	R^2 (trend deviation, ↑)	RMSE (↓)	R^2 (↑)	Corr (↑)
Hierarchical CNN	0.204 ± 0.024	-0.145 ± 0.199	21.075 ± 1.712	0.708 ± 0.025	0.846 ± 0.016
FCN	0.183 ± 0.013	0.067 ± 0.124	19.801 ± 1.647	0.743 ± 0.023	0.866 ± 0.014
Inception	*0.167 ± 0.014*	*0.222 ± 0.150*	*17.542 ± 1.300*	*0.798 ± 0.021*	**0.901 ± 0.012**
TST	0.217 ± 0.029	-0.302 ± 0.244	23.044 ± 2.684	0.651 ± 0.058	0.819 ± 0.039
Per timestep MLP	0.172 ± 0.013	0.183 ± 0.093	18.217 ± 1.221	0.781 ± 0.027	0.889 ± 0.012
LSTM + Attn	0.171 ± 0.014	0.191 ± 0.119	17.740 ± 0.991	0.793 ± 0.018	0.896 ± 0.012
LADAA	**0.164 ± 0.008**	**0.258 ± 0.108**	**17.012 ± 1.067**	**0.809 ± 0.029**	**0.901 ± 0.017**

Table 2: Results on crop yield forecasting (from Aug. 1). Average and standard deviation across 5 test folds. "Trend deviation" means we fit a line (year vs. yield) for each county to model technological improvements, and try to predict the fraction deviation from the trend. Evaluation metrics are defined in the Appendix.

from the Monash Time Series Extrinsic Regression archive (Tan et al. 2020). These include diverse tasks such as predicting the energy usage of a home, predicting air quality given climate information, and predicting human heart rate from PPG sensors. The evaluation metric is root mean squared error (RMSE) (see appendix for definition). We compare against a wide range of baselines used in (Zerveas et al. 2021). In Table 1 we include top five-ranking baselines reported in (Zerveas et al. 2021), which include all methods that performed in the top two on any dataset:

- **Random Forest** (Breiman 2001) and **XGBoost** (Chen et al. 2015) are tree-based methods for tabular data. For these, the multivariate time series is flattened into a single feature vector (ignoring temporal structure).
- **Rocket** (Dempster, Petitjean, and Webb 2020) slides random convolutional filters down the time series, applies global max pooling to extract features, and trains a linear model on top.
- **InceptionTime** (Ismail Fawaz et al. 2020) is a convolutional network with an ensemble of different filter sizes.
- **Time Series Transformer (TST)** (Zerveas et al. 2021) is a Transformer model adapted for time series. Since we are evaluating our proposed architectures, we only compare against the supervised-only model, and do not consider the unsupervised pretraining task.

Methods that we add to the benchmark include:

- **Per-Timestep MLP**, defined in section 3.1.
- **LSTM+Attn**, a recurrent neural network followed by attention pooling, which dominated natural language processing before Transformers (Bahdanau 2014).
- **ConViT** (d'Ascoli et al. 2021), a similar approach that injects locality inductive biases in transformers for vision. However, it only uses *linear* decays for the relative positional encoding, and combines content and positional attention via gating (instead of only using positional attention).
- **ConvTran** (Foumani et al. 2024), the SOTA method in time series *classification*, which adds a learnable relative position bias to the (post-softmax) content attention, but does not initialize the biases to behave like a convolution.
- **LADAA** is our method defined in Sections 3.3-3.4, where attention depends solely on relative distances.

For new methods, following (Zerveas et al. 2021) we split the train split into 80% train and 20% validation, perform a small hyperparameter search, and choose hyperparameters that performed best on the validation set. We then retrain the model on the entire train split and report the test split performance. See Appendix for details.

The results are shown in Table 1. Impressively, the per-timestep MLP already outperforms all the methods reported in (Zerveas et al. 2021), on all datasets except for the IEEEPPG dataset, which has the most timesteps and includes intricate features that cannot be captured in small windows. Across the six datasets, per-timestep MLP reduces RMSE by an average of 9.1% over the strongest baseline (TST). The fact that such a simple architecture can outper-

form state-of-the-art Transformer and convolutional models shows that many time series extrinsic regression tasks are separable into timestep effects, and self-attention is not adding much. LSTM+Attn is also strong, beating all baselines on all datasets (reducing RMSE by an average of 3.6% over TST), performing similarly to per-timestep MLP in many cases. This makes sense; an LSTM can emulate a per-timestep MLP if the cell "forgets" the input from previous timesteps, and only uses the current timestep's input.

ConViT and ConvTran also outperform Time Series Transformer on most datasets, indicating that using relative distances when computing attention can be helpful. However, LADAA does even better than these methods and achieves a new state-of-the-art on all datasets,[2] reducing RMSE by an average of 16.7% over TST. The gain is statistically significant ($p < 0.05$) on all datasets, based on a paired t-test (see Appendix). Thus, content attention seems to be useless and even somewhat harmful. *Completely replacing* content attention with relative-positional attention (instead of combining them together), and initializing the model to behave convolutionally, can substantially improve results over standard Transformers and other deep models.

4.2 Crop Yield Forecasting experiments

As a real-world application, consider the task of forecasting crop yields from time series of daily weather and management data. For each year and US county, our task is to forecast the corn yield well before harvest, given daily weather data from January 1 – July 31 (which is challenging as most corn is harvested in autumn). Here we test different versions of the *temporal encoder*, while keeping the encoders for non-temporal data constant. Since the temporal encoder is a part of a larger neural model, we can only compare against differentiable neural networks (Inception, TST) and not classic methods (Random Forest, XGBoost, Rocket). We add two more convolutional baselines: **Hierarchical CNN** from (Fan et al. 2022) and **Fully-Convolutional Network** (Long, Shelhamer, and Darrell 2015). We use the dataset from (Fan et al. 2022) but substantially modify the evaluation and features, so the numbers are not comparable. We perform 5-fold cross-validation, splitting the data into folds by year. For each round we hold out one fold for test and one fold for validation (early stopping and tuning).

The original Time Series Transformer performed very poorly as its lack of temporal inductive biases leads to overfitting. Hierarchical CNN performed fairly poorly, perhaps because it was crafted to extract most signal from the middle of the time series (while in this early prediction task, the most relevant information is at the end). Inception, LSTM+Attention, Per-Timestep MLP, and LADAA all performed competitvely, showing that learning *local features* before aggregating is important. LADAA and Inception perform best; both allow learning local features with a wide range of receptive fields, but LADAA does slightly better, perhaps because it can learn the optimal receptive fields. LADAA performs the best across all metrics.

[2]To be fair, on BenzeneConcentration, per-timestep MLP is extremely close, perhaps because most of the signal is in the last timestep. Also, LiveFuel is problematic; predicting the mean label and ignoring the input achieves an RMSE of 41.135, virtually state-of-the-art.

	Benzene	IEEEPPG
Only position	*0.320 ± 0.030*	**21.831 ± 0.717**
After gating	0.346 ± 0.070	*22.807 ± 0.142*
After	2.812 ± 0.299	32.573 ± 0.215
Before	**0.269 ± 0.030**	26.654 ± 0.245
Only content	0.378 ± 0.084	31.052 ± 1.209

Table 3: Ablation: how to combine position and content attention After gating means we combine using a weighted sum $(1-\lambda)\cdot content + \lambda \cdot pos$, where λ is learned.

	Benzene	IEEEPPG
Convalibi init	*0.320 ± 0.030*	**21.831 ± 0.717**
Zero init	0.550 ± 0.031	*22.709 ± 0.296*
Alibi init	**0.318 ± 0.029**	30.285 ± 0.690
Convit init	0.416 ± 0.117	22.786 ± 0.535
Convit (learned slope/offset)	0.324 ± 0.045	23.053 ± 1.551

Table 4: Ablation: LADAA Initialization

4.3 Ablations

In Table 3 we check if we can get a better result by combining content and positional attention (either after/before the softmax, or after softmax with gating). "Only position" still seems best on average, indicating that attention scores should primarily depend on position (not content). "After gating" also does well, likely because we initialized it to favor position, following (d'Ascoli et al. 2021). In Table 4 we check if the convolutional initialization of LADAA was necessary. We find that Convalibi init outperforms zero init, showing that initializing the bias table to focus on nearby timesteps was helpful. The remaining methods performed similarly; the exact form of the initialization may not be crucial as long as it encourages locality. We further show this in Table 7 (Appendix), which shows that even for vanilla Transformers, restricting attention to a local neighborhood via masking improves performance on time series extrinsic regression. More ablations and hyperparameter sensitivity tests can be found in the Appendix.

5 Conclusion

We have shown that vanilla self-attention is not well-suited for time series extrinsic regression; even a lightweight per-timestep MLP achieves strong performance, underscoring the importance of localized modeling. Further, making attention scores depend *only* on relative distances, while initializing them to focus on nearby timesteps, can improve performance even further. Building on these insights, we introduced LADAA, a lightweight, locality-and-distance-aware attention mechanism that further improves performance by relying solely on relative positional distances and by biasing attention toward nearby timesteps. Our extensive empirical analysis shows that LADAA consistently outperforms all baselines across diverse benchmarks, includ-

ing a challenging real-world application, concerning the task of forecasting crop yields from daily weather time series. Our results highlight that incorporating local structure and distance-aware attention is critical for effective time series modeling. LADAA thus sets a new state-of-the-art and provides a foundation for future work on structured attention in time series extrinsic regression tasks.

Acknowledgments

Redacted for anonymity.

References

Bahdanau, D. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

Breiman, L. 2001. Random forests. *Machine learning*, 45(1): 5–32.

Butler, E. E.; and Huybers, P. 2015. Variations in the sensitivity of US maize yield to extreme temperatures by region and growth phase. *Environmental Research Letters*, 10(3): 034009.

Candanedo, L. M.; Feldheim, V.; and Deramaix, D. 2017. Data driven prediction models of energy use of appliances in a low-energy house. *Energy and buildings*, 140: 81–97.

Chen, T.; He, T.; Benesty, M.; Khotilovich, V.; Tang, Y.; Cho, H.; Chen, K.; Mitchell, R.; Cano, I.; Zhou, T.; et al. 2015. Xgboost: extreme gradient boosting. *R package version 0.4-2*, 1(4): 1–4.

Chowdhury, R. R.; Zhang, X.; Shang, J.; Gupta, R. K.; and Hong, D. 2022. Tarnet: Task-aware reconstruction for time-series transformer. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 212–220.

Cordonnier, J.-B.; Loukas, A.; and Jaggi, M. 2019. On the relationship between self-attention and convolutional layers. *arXiv preprint arXiv:1911.03584*.

De Vito, S.; Massera, E.; Piga, M.; Martinotto, L.; and Di Francia, G. 2008. On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B: Chemical*, 129(2): 750–757.

Dempster, A.; Petitjean, F.; and Webb, G. I. 2020. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. *Data Mining and Knowledge Discovery*, 34(5): 1454–1495.

d'Ascoli, S.; Touvron, H.; Leavitt, M. L.; Morcos, A. S.; Biroli, G.; and Sagun, L. 2021. Convit: Improving vision transformers with soft convolutional inductive biases. In *International conference on machine learning*, 2286–2296. PMLR.

Fan, J.; Bai, J.; Li, Z.; Ortiz-Bobea, A.; and Gomes, C. P. 2022. A GNN-RNN approach for harnessing geospatial and temporal information: application to crop yield prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, 11873–11881.

Foumani, N. M.; Tan, C. W.; Webb, G. I.; and Salehi, M. 2024. Improving position encoding of transformers for multivariate time series classification. *Data mining and knowledge discovery*, 38(1): 22–48.

Gulati, A.; Qin, J.; Chiu, C.-C.; Parmar, N.; Zhang, Y.; Yu, J.; Han, W.; Wang, S.; Zhang, Z.; Wu, Y.; et al. 2020. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*.

Guo, J.; Han, K.; Wu, H.; Tang, Y.; Chen, X.; Wang, Y.; and Xu, C. 2022. Cmt: Convolutional neural networks meet vision transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 12175–12185.

Hassani, A.; Walton, S.; Shah, N.; Abuduweili, A.; Li, J.; and Shi, H. 2021. Escaping the big data paradigm with compact transformers. *arXiv preprint arXiv:2104.05704*.

Ismail Fawaz, H.; Lucas, B.; Forestier, G.; Pelletier, C.; Schmidt, D. F.; Weber, J.; Webb, G. I.; Idoumghar, L.; Muller, P.-A.; and Petitjean, F. 2020. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 34(6): 1936–1962.

Kim, D.; Park, J.; Lee, J.; and Kim, H. 2024. Are self-attentions effective for time series forecasting? *Advances in Neural Information Processing Systems*, 37: 114180–114209.

Li, S.; You, C.; Guruganesh, G.; Ainslie, J.; Ontanon, S.; Zaheer, M.; Sanghai, S.; Yang, Y.; Kumar, S.; and Bhojanapalli, S. 2023. Functional interpolation for relative positions improves long context transformers. *arXiv preprint arXiv:2310.04418*.

Liu, M.; Ren, S.; Ma, S.; Jiao, J.; Chen, Y.; Wang, Z.; and Song, W. 2021a. Gated transformer networks for multivariate time series classification. *arXiv preprint arXiv:2103.14438*.

Liu, S.; Yu, H.; Liao, C.; Li, J.; Lin, W.; Liu, A. X.; and Dustdar, S. 2022. Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting. In *International Conference on Learning Representations*.

Liu, Y.; Hu, T.; Zhang, H.; Wu, H.; Wang, S.; Ma, L.; and Long, M. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*.

Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; and Guo, B. 2021b. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, 10012–10022.

Long, J.; Shelhamer, E.; and Darrell, T. 2015. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 3431–3440.

Mohammadi Foumani, N.; Miller, L.; Tan, C. W.; Webb, G. I.; Forestier, G.; and Salehi, M. 2024. Deep learning for time series classification and extrinsic regression: A current survey. *ACM Computing Surveys*, 56(9): 1–45.

Nie, Y.; Nguyen, N. H.; Sinthong, P.; and Kalagnanam, J. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*.

Ortiz-Bobea, A. 2021. The empirical analysis of climate change impacts and adaptation in agriculture. In *Handbook of agricultural economics*, volume 5, 3981–4073. Elsevier.

Park, N.; and Kim, S. 2022. How Do Vision Transformers Work? In *International Conference on Learning Representations*.

Peng, Z.; Huang, W.; Gu, S.; Xie, L.; Wang, Y.; Jiao, J.; and Ye, Q. 2021. Conformer: Local features coupling global representations for visual recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, 367–376.

Press, O.; Smith, N. A.; and Lewis, M. 2021. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*.

Shaw, P.; Uszkoreit, J.; and Vaswani, A. 2018. Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*.

Tan, C. W.; Bergmeir, C.; Petitjean, F.; and Webb, G. I. 2020. Monash university, uea, ucr time series extrinsic regression archive. *arXiv preprint arXiv:2006.10996*.

Tan, C. W.; Bergmeir, C.; Petitjean, F.; and Webb, G. I. 2021. Time series extrinsic regression: Predicting numeric values from time series data. *Data Mining and Knowledge Discovery*, 35(3): 1032–1060.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Wen, Q.; Zhou, T.; Zhang, C.; Chen, W.; Ma, Z.; Yan, J.; and Sun, L. 2023. Transformers in time series: a survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 6778–6786.

Wu, H.; Xiao, B.; Codella, N.; Liu, M.; Dai, X.; Yuan, L.; and Zhang, L. 2021a. Cvt: Introducing convolutions to vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, 22–31.

Wu, H.; Xu, J.; Wang, J.; and Long, M. 2021b. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34: 22419–22430.

Zeng, A.; Chen, M.; Zhang, L.; and Xu, Q. 2023. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, 11121–11128.

Zerveas, G.; Jayaraman, S.; Patel, D.; Bhamidipaty, A.; and Eickhoff, C. 2021. A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2114–2124.

Zhang, Y.; and Yan, J. 2023. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *The eleventh international conference on learning representations*.

Zhang, Z.; Pi, Z.; and Liu, B. 2014. TROIKA: A general framework for heart rate monitoring using wrist-type photoplethysmographic signals during intensive physical exercise. *IEEE Transactions on biomedical engineering*, 62(2): 522–531.

Zhou, H.; Zhang, S.; Peng, J.; Zhang, S.; Li, J.; Xiong, H.; and Zhang, W. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, 11106–11115.

Zhou, T.; Ma, Z.; Wen, Q.; Wang, X.; Sun, L.; and Jin, R. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*, 27268–27286. PMLR.

A Technical Appendices

A.1 Updated Table 1

We fixed a few minor discrepancies in Table 1 - in particular, we used an incorrect (suboptimal) patch size on IEEEPPG and LiveFuelMoistureContent. Table 5 is a revised version of Table 1, which does not significantly alter the conclusions. While LADAA is no longer the best method on LiveFuelMoistureContent, it still outperforms all previous baselines, and this dataset is problematic (simply predicting the average value gives an RMSE of 41.135, which is already virtually state-of-the-art).

A.2 Statistical tests

We conduct a statistical test using a paired t-test, comparing LSTM+attn, per-timestep MLP, and LADAA with the original Time Series Transformer (TST) (Zerveas et al. 2021). Since TST did not provide individual runs, we did our best to reproduce their results. We only did this analysis on 3 seeds; we expect lower p-values with more runs. Nevertheless, LADAA's improvement over the strongest baseline (TST) is statistically significant at the $p < 0.05$ level for all datasets (except LiveFuel), and exceeds the threshold by a large margin on many datasets.

A.3 Local Mask

To test the impact of locality, we also ran a test with a "local mask" on the vanilla Transformer from (Zerveas et al. 2021) (no relative positional encoding). Specifically, we mask out all entries at indices (i, j) in the attention matrix where $|i - j| > D$ (where D is a distance threshold), by setting their pre-softmax attention scores to $-\infty$ (so they get zero attention after softmax). A local mask of 0 means that each timestep only pays attention to itself, and is similar to a per-timestep MLP. For all of these models, we added "attention pooling" at the end (instead of the default linear pooling) so that attention can still be used for global aggregation.

We can see results in Table 7. For Appliances and BeijingPM10, having a local mask outperforms no mask. Forcing timesteps to only pay attention within a small neighborhood can actually provide a valuable inductive bias that helps performance; the longer-range attention scores did not really help. For Benzene, no mask does best in this experiment, but local mask 0's performance is almost identical, indicating that the between-timestep interactions are not very important.

A.4 Dataset statistics

We summarize the seven datasets used in the paper in Table 8. The datasets encompass a wide range of temporal resolutions and sequence lengths, data sizes, and variable numbers.

A.5 Full Loss Function

All models are trained primarily using MSE (Mean Squared Error loss), plus a combination of attention smoothness (Eq. 18) and L1 regularization losses:

$$\mathcal{L} = \sum_{i=1}^{N}(y_i - \hat{y}_i^2) + \lambda_1 \sum_{w \in weights} |w| \tag{19}$$

$$+ \lambda_2 \sum_{layer,head} \mathcal{L}_{smooth}[layer, head] \tag{20}$$

where y_i is the ground-truth label for example i, and $\hat{y}_i$ is the model's prediction for example i. 'weights' considers all the linear-layer weights in the network but excludes biases.

A.6 Evaluation Metrics

Our main evaluation metric is Root Mean Squared Error (RMSE), which is defined as

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{21}$$

where y_i is the ground-truth label for example i, and $\hat{y}_i$ is the model's prediction for example i. R^2 is defined as

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y}_i)^2} \tag{22}$$

where $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$ is the average of all labels.

Correlation coefficient (Pearson) is defined as

$$Corr = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{23}$$

Again $\bar{x}$ and $\bar{y}$ are the means of x and y respectively. We let x be the model prediction and y be the true label.

A.7 Hyperparameter Tuning

Note that the time series extrinsic regression datasets only provide a train/test split, and not a validation split. All codebases that implement baseline methods (Tan et al. 2021; Zerveas et al. 2021; Chowdhury et al. 2022) implicitly use the test set as a validation set for early stopping. While this is not best practice, it is difficult to avoid as there is a significant distribution shift between the train and test sets provided in (Tan et al. 2020). If we were to hold out a portion of the train set as a validation set, the validation performance is not predictive of the test performance and is a poor guide for model selection.

That being said, for hyperparameter tuning, we still split the "train" split into 80% training, and 20% validation, and selected based on validation performance. In cases where validation and test performances were poorly correlated (AppliancesEnergy, BenzeneConcentration, IEEEPPG, LiveFuelMoistureContent), we held out the *last* 20% of the train split, which often represents a validation set that overlaps less with the train set (either in terms of time block, or representing different patience in the case of IEEEPPG (Zhang, Pi, and Liu 2014)). However, we peeked at the test performance occasionally during method development.

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG
Random Forest	3.455	0.855	94.072	63.301	44.657	32.109
XGBoost	3.489	0.637	93.138	59.495	44.295	31.487
Rocket	2.299	3.36	120.057	62.769	41.829	36.515
Inception	4.435	1.584	96.749	62.227	51.539	*23.903*
TST	2.228	0.517	91.344	60.357	42.607	25.042
Per timestep MLP	*1.954 ± 0.133*	*0.295 ± 0.052*	89.363 ± 1.195	55.798 ± 0.999	**41.138 ± 0.010**	32.305 ± 0.952
LSTM + Attn	2.208 ± 0.104	0.423 ± 0.006	90.519 ± 0.957	57.059 ± 0.405	*41.487 ± 0.217*	24.784 ± 0.630
ConViT	2.398 ± 0.069	0.326 ± 0.107	*87.685 ± 0.789*	*54.759 ± 0.744*	42.842 ± 0.452	*23.188 ± 0.888*
ConvTran	2.101 ± 0.264	0.536 ± 0.048	89.230 ± 0.610	56.847 ± 0.306	43.450 ± 0.745	24.997 ± 1.271
LADAA	**1.685 ± 0.084**	**0.273 ± 0.034**	**86.878 ± 1.051**	**53.999 ± 1.215**	41.683 ± 0.839	**20.937 ± 0.140**

Table 5: Update to Table 1, fixing the patch size used in IEEEPPG and LiveFuelMoistureContent. Results (RMSE ↓) on Time Series Extrinsic Regression archive. Above the line are baseline results reported in (Zerveas et al. 2021). Below the line is our methods; we provide mean and standard deviation over 3 seeds. Best is **bolded**, second-best is *italicized*. Per-timestep MLP outperforms all above baselines for all datasets except IEEEPPG. LSTM+Attn outperforms all baselines. ConViT and Convtran are mixed but perform well overall. LADAA is the best on all datasets.

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG	CropYield
Baseline: TST	2.324 ± 0.168	0.728 ± 0.134	91.528 ± 0.971	60.348 ± 0.996	43.656 ± 0.137	29.869 ± 0.476	23.044 ± 2.684
Per timestep MLP	1.954 ± 0.133	*0.295 ± 0.052*	*89.363* ± 1.195	*55.798 ± 0.999*	**41.138 ± 0.010**	32.305 ± 0.952	18.217 ± 1.221
	p=0.038*	p=0.050*	p=0.222	p=0.054	p=0.001***	-	p=0.003**
LSTM + Attn	2.208 ± 0.104	0.423 ± 0.006	90.519 ± 0.957	57.059 ± 0.405	*41.487 ± 0.217*	24.784 ± 0.630	17.740 ± 0.991
	p=0.448	p=0.057	p=0.219	p=0.051	p=0.004**	p=0.015*	p=0.003**
LADAA	**1.685 ± 0.084**	**0.273 ± 0.034**	**86.878 ± 1.051**	**53.999 ± 1.215**	41.643 ± 0.839	**20.937 ± 0.140**	**17.012 ± 1.066**
	p=0.031*	p=0.031*	p=0.000***	p=0.028*	p=0.059	p=0.000***	p=0.002**

Table 6: Statistical analysis. Because (Zerveas et al. 2021) do not provide individual run results, we show our best attempt to reproduce their results using their code. For each method, we show RMSE mean ± sample standard deviation (dividing by $N-1$). The p-value is from a paired t-test comparing the method's results with the TST baseline's results. Each entry only uses 3 seeds due to computational limitations (we expect lower p-values if we were able to run more). *: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$.

Mask dist.	Appliances	Benzene	BeijingPM10
None	2.494 ± 0.088	0.483 ± 0.020	91.076 ± 0.461
0	2.023 ± 0.097	0.486 ± 0.058	90.828 ± 0.466
2	**1.968 ± 0.133**	0.806 ± 0.109	90.590 ± 1.572
4	2.077 ± 0.085	0.729 ± 0.142	**89.738 ± 1.656**
8	2.129 ± 0.039	0.947 ± 0.329	90.338 ± 1.114

Table 7: Local mask test (original Time Series Transformer + attention pooling). A mask of 2 means that each timestep can pay attention to at most 2 timesteps away. Using attention pooling, adding at start.

For the crop yield dataset, we split the years into 5 folds. For each fold, we hold it out as test, and select another one to be validation. We tune hyperparameters and early stopping on one validation fold. Then we use the best hyperparameters on all 5 folds and compute the average test-fold results.

We used the RAdam optimizer for the Time Series Extrinsic Regression datasets (following (Zerveas et al. 2021)), and the Adam optimizer for crop yield prediction. We generally tuned learning rate from $[10^{-4}, 10^{-3}, 10^{-2}]$, L1 regularization from $[0, 10^{-4}, 10^{-3}, 10^{-2}]$, attention smoothness from $[0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}]$. For Convit slope, we considered values from $[0.1, 0.25, 0.5, 1.0]$ on a few datasets, and decided on $\alpha = 0.25$, which worked for all datasets. For IEEEPPG and LiveFuelMoistureContent, the number of timesteps is very large, so we needed to use patching for the model to fit in GPU memory. We selected patch size and stride from $[(16, 8), (8, 4), (4, 4)]$.

The final parameters selected are shown in Tables 9, 10, 11, 12.

Tables 13, 14, 15, 16, provide sensitivity analyses on how much changing certain hyperparameters impact the final test performance (holding all other hyperparameters constant, although we did not always start from the optimal set of hyperparameters). In general, the results seem fairly robust to a wide range of hyperarameters. While they do fluctuate a little bit, even the worse options are still better than the strongest baseline.

A.8 Additional ablations

In Table 17 we considered different ways to add the absolute positional encoding. Zerveas et al. (2021) added the positional encoding at the start, with uniform initialization. However, in our model, we seem to find that concatenating the absolute positional encoding right before attention pooling does a little better, potentially because the initial relative distance attention layers are just learning local features and the absolute position would distract it; absolute position information is most important at the final pooling. Sinusoidal initialization also seemed to give a slight benefit over random initialization (at least on Benzene and IEEEPPG).

Dataset	Train Size	Test Size	Length (timesteps)	Num variables
AppliancesEnergy	96	42	144	24
BenzeneConcentration	3433	5445	240	8
BeijingPM10Quality	12432	5100	24	9
BeijingPM25Quality	12432	5100	24	9
LiveFuelMoistureContent	3493	1510	365	7
IEEEPPG	1768	1328	1000	5
CropYield	57892	14430	212	238

Table 8: Summary of datasets used.

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG	CropYield
Patch size/stride	1/1	1/1	1/1	1/1	4/4	8/4	4/2
Batch size	16	128	128	128	128	128	64
Learning rate	1e-2	1e-3	1e-3	1e-3	1e-2	1e-2	1e-3
L1 regularization λ_1	1e-3	0	1e-2	0	1e-4	1e-3	0
Convit slope α	0.25	0.25	0.25	0.25	0.25	0.25	0.25
Attention smoothing λ_2	1e-3	1e-1	1e-2	1e-2	1e-1	1e-2	1e-1

Table 9: LADAA hyperparameters.

In Table 18 we show that attention pooling is better than average pool or linear pool in most cases with LADAA. One possible reason may be that attention pool is capable of performing a max pool operation (see proof below), or some timesteps may have a disproportionate impact based on their features.

A.9 Crop Yield Model Summary

For the crop yield dataset, each datapoint represents a county (location) c at a given $year$. For corn, we have a total of 76637 datapoints. At each datapoint, we have the yield for that year $y_{c,year}$, as well as:

- Time-series data: ($\mathbf{x}_{c,year}^{(weather)} \in \mathbb{R}^{T' \times V}$, such as weather and management data. For each variable, we have a value for each day in the forecast period (Jan. 1-July 31 of that year)
- County-climate data: the average weather data for that county across all years, for each day in the year ($\mathbf{x}_{c}^{(climate)} \in \mathbb{R}^{T \times V}$)
- Other fixed county features $\mathbf{x}_{c}^{(soil)}$, such as soil characteristics.

We encode the weather and county-climate features using the temporal encoder, then create a "county embedding" from the county-climate embedding and the other county features, then finally combine everything with fully-connected layers.

$$\mathbf{h}_{c,year}^{(weather)} = TemporalEnc(\mathbf{x}_{c,year}^{(weather)}) \quad (24)$$

$$\mathbf{h}_{c}^{(climate)} = TemporalEnc(\mathbf{x}_{c}^{(climate)}) \quad (25)$$

$$\mathbf{h}_{c}^{(county)} = MLP(\mathbf{h}_{c}^{(climate)}, \mathbf{x}_{c}^{(soil)}, year) \quad (26)$$

$$\hat{y} = MLP(\mathbf{h}_{c,year}^{(weather)}, \mathbf{h}_{c}^{(county)}) \quad (27)$$

For simplicity, we treat each county/year as a distinct data point, as this already achieves good performance, instead of considering correlations between counties and years as done in (Fan et al. 2022).

A.10 Computational resources

We ran all experiments on a single NVIDIA V100 GPU. Each run usually took no more than 3 hours on the most expensive datasets (typically less for most datasets). We reserved about 20GB of CPU memory for the time series extrinsic regression datasets, and about 100GB for the crop yield dataset (as the data file is quite large).

A.11 Proofs

Expanded proof of Proposition 1:

Proposition 1. *For any 1D convolution with filter size h, an "Only Relative Positional Encoding" layer with h heads is capable of expressing the same function.*

Proof. Suppose we have a convolutional filter that at timestep i, takes inputs at positions $i - j$ for various offsets $j \in \mathcal{N}$. We simply make each attention head focus on a particular offset $(i - j)$, by setting $\mathbf{B}_{i-j}^{h}$ to a large value and $\mathbf{B}_{k \neq (i-j)} = -\infty$, so that after the softmax, the head has attention score 1 on that particular offset and 0 elsewhere. Thus, each head is simply extracting the features at timestep $i - j$.

To proceed, we can directly apply Lemma 1 from (Cordonnier, Loukas, and Jaggi 2019), which we restate here for completeness.

Note that self-attention can be written as follows (where the input $\mathbf{X} \in \mathbb{R}^{T \times D_{in}}$ and output is $\mathbb{R}^{T \times D_h}$):

$$\text{SelfAttn}(\mathbf{X})_{t,:} = \text{softmax}(\mathbf{A}_{t,:})\mathbf{X}\mathbf{W}_{val} \quad (28)$$

where the attention matrix $\mathbf{A} \in \mathbb{R}^{T \times T}$ is

$$\mathbf{A} := \mathbf{X}\mathbf{W}_{qry}\mathbf{W}_{key}^{T}\mathbf{X}^{T} \quad (29)$$

Now, *multihead* self-attention is

$$\text{MHSA}(\mathbf{X}) = \text{concat}_{h \in [N_h]}[\text{SelfAttn}_h(\mathbf{X})]\mathbf{W}_{out} + \mathbf{b}_{out} \quad (30)$$

where $\mathbf{W}_{out} \in \mathbb{R}^{N_h D_h \times D_{out}}$, $\mathbf{b}_{out} \in \mathbb{R}^{D_{out}}$.

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG	CropYield
Per-Timestep MLP: learning rate	1e-2	1e-3	1e-4	1e-4	1e-4	1e-4	1e-3
LSTM: learning rate	1e-3	1e-2	1e-4	1e-3	1e-2	1e-3	1e-3

Table 10: Per-Timestep MLP and LSTM hyperparameters. Patch/stride are same as LADAA.

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG
Batch size	16	128	128	128	128	32
Learning rate	1e-2	1e-2	1e-3	1e-3	1e-2	1e-4
Number of heads	16	8	8	8	8	16

Table 11: ConViT hyperparameters.

We can rewrite this (by decomposing the matrix multiplication over the concatenated column-blocks) as

$$\text{MHSA}(\mathbf{X}) = \mathbf{b}_{out} + \sum_{h \in [N_h]} [\text{SelfAttn}_h(\mathbf{X})] \mathbf{W}_{val}^{(h)} \mathbf{W}_{out}[\text{block } h] \tag{31}$$

where "block h" represents rows of $\mathbf{W}_{out}$ that will hit the relevant columns of $\text{SelfAttn}_h(x)$ generated by head h, e.g. the range $[(h-1)D_h + 1, hD_h + 1)$. We can replace the pair of matrices $(\mathbf{W}_{val}^{(h)}, \mathbf{W}_{out}[\text{block } h])$ with a learned matrix $\mathbf{W}^{(h)}$ for each head. Now consider one output pixel i:

$$\text{MHSA}(\mathbf{X})_{i,:} = \mathbf{b}_{out} + \sum_{h \in [N_h]} \left(\sum_j \text{softmax}(\mathbf{A}_{i,:}^{(h)})_j \mathbf{X}_{j,:} \right) \mathbf{W}^{(h)} \tag{32}$$

But we have established that we can make the head have attention score 1 at a particular offset $i - j(h)$ and zero elsewhere. Thus, the layer's output at pixel q is

$$\text{MHSA}(\mathbf{X})_i = \mathbf{b}_{out} + \sum_{h \in [N_h]} \mathbf{X}_{i-j(h),:} \mathbf{W}^{(h)} \tag{33}$$

This is now exactly equivalent to a convolutional layer that pays attention to N_h offsets j. □

We also prove a simple property about attention pooling[3]:

Proposition 2. *Attention pooling can learn an average or max pooling operation over timesteps.*

Proof. The average pooling case is simple; MLP_{attn} just outputs $\frac{1}{T}$ for all timesteps using a bias (ignoring the timestep features). For max pooling, suppose that each timestep has a feature x_i, and we want to extract the timestep with the max value of x_i. Let $\hat{x} = \max(x_i)$. We have MLP_{attn} output cx_i, and let $c \to \infty$. Then, after softmax,

$$\begin{aligned}\alpha_i &= \lim_{c\to\infty} \frac{\exp(cx_i)}{\sum_j \exp(cx_j)} \\ &= \lim_{c\to\infty} \frac{\exp(c(x_i - \hat{x}))}{\sum_j \exp(c(x_j - \hat{x}))} \quad \text{(Multiply top/bottom by } e^{-c\hat{x}}\text{)}\end{aligned}$$

Now, if $x_i = \hat{x}_i$ (the max element), $\exp(c(x_i - \hat{x})) = \exp(0) = 1$:

$$\begin{aligned}\alpha_i &= \lim_{c\to\infty} \frac{1}{1 + \sum_{x_j \neq \hat{x}} \exp(c(x_j - \hat{x}))} \\ &= \frac{1}{1+0} = 1\end{aligned}$$

since $(x_j - \hat{x})$ is always negative if $x_j \neq \hat{x}$, as $\hat{x}$ is the maximum. Thus, as $c \to \infty$, the term inside the exponential goes to $-\infty$, so the exponential goes to 0.

For other entries $x_i \neq \hat{x}$:

$$\begin{aligned}\alpha_i &= \lim_{c\to\infty} \frac{\exp(c(x_i - \hat{x}))}{1 + \sum_{x_j \neq \hat{x}} \exp(c(x_j - \hat{x}))} \\ &= \frac{0}{1+0} = 0\end{aligned}$$

again because the $(x_i - \hat{x})$ terms are all negative. Thus, the attention weight will be 1 on the max element, and zero everywhere else. □

[3]Refer: https://math.stackexchange.com/questions/2656231/proving-that-softmax-converges-to-argmax-as-we-scale-x

	Appliances	Benzene	BeijingPM10	BeijingPM25	LiveFuel	IEEEPPG
Learning rate	1e-2	1e-3	1e-3	1e-3	1e-3	1e-4
Batch size	16	128	128	128	128	32
Number of heads	8	8	16	16	16	8

Table 12: ConvTran hyperparameters.

	IEEEPPG	LiveFuel
16/8	22.664 ± 0.369	42.461 ± 0.564
8/4	21.964 ± 0.667	42.256 ± 0.896
4/4	21.894 ± 0.180	42.552 ± 0.330

Table 13: Sensitivity analysis: Patch size/stride.

	Benzene	BeijingPM10	IEEEPPG
Best baseline	0.517	91.344	23.902
$\lambda_1 = 0$	0.289 ± 0.002	86.725 ± 1.683	21.816 ± 0.526
$\lambda_1 = 10^{-4}$	0.292 ± 0.036	87.072 ± 1.568	21.658 ± 1.273
$\lambda_1 = 10^{-3}$	0.297 ± 0.040	87.101 ± 2.344	22.188 ± 0.380
$\lambda_1 = 10^{-2}$	0.266 ± 0.027	86.878 ± 1.051	22.535 ± 0.900

Table 14: Sensitivity analysis: L1 regularization

	Benzene	BeijingPM10	IEEEPPG
Best baseline	0.517	91.344	23.902
$\alpha = 0.01$	0.383 ± 0.057	86.755 ± 1.513	21.966 ± 0.801
$\alpha = 0.1$	0.326 ± 0.029	87.857 ± 0.778	22.324 ± 1.100
$\alpha = 0.25$	0.297 ± 0.040	86.878 ± 1.051	22.535 ± 0.900
$\alpha = 0.5$	0.314 ± 0.062	86.978 ± 1.467	21.932 ± 0.577
$\alpha = 1$	0.304 ± 0.055	86.468 ± 1.469	21.098 ± 0.665

Table 15: Sensitivity analysis: initial convit slope. Higher slope means that attention is more focused on a single point, lower slope means that attention is more spread out

	Benzene	BeijingPM10	IEEPPG
Best baseline	0.517	91.344	23.902
$\lambda_2 = 0$	0.282 ± 0.014	87.843 ± 1.194	21.586 ± 1.477
$\lambda_2 = 10^{-3}$	0.273 ± 0.019	87.783 ± 1.156	21.340 ± 0.649
$\lambda_2 = 10^{-2}$	0.297 ± 0.040	86.878 ± 1.051	22.763 ± 0.942
$\lambda_2 = 10^{-1}$	0.277 ± 0.037	86.558 ± 1.111	22.535 ± 0.900

Table 16: Sensitivity analysis: attention smoothness

	Benzene	BeijingPM10	IEEEPPG
Concat before pool - sin init	$\mathbf{0.320 \pm 0.030}$	$\mathbf{88.351 \pm 1.293}$	21.831 ± 0.717
Add at start - sin init	0.374 ± 0.065	90.290 ± 0.955	$\mathbf{21.004 \pm 0.428}$
Add at start - uniform init	0.474 ± 0.057	89.616 ± 0.280	22.638 ± 0.147

Table 17: Ablation: Absolute Positional Encoding

	Benzene	BeijingPM10	IEEEPPG
Attention pool	$\mathbf{0.273 \pm 0.034}$	$\mathbf{86.878 \pm 1.051}$	$\mathbf{22.535 \pm 0.900}$
Average pool	0.566 ± 0.076	88.047 ± 0.204	23.238 ± 2.307
Linear pool	0.418 ± 0.068	88.453 ± 0.757	23.433 ± 1.108

Table 18: Ablation: Pooling type for LADAA.